

CityLife: Dynamic Coastal City and Urban Scenes in OpenGL

A Multi-Scenario 2D Urban Simulation
Computer Graphics Final Project

Group Members

Name	Student ID	Contribution Area	Contribution (%)
Sabit Hassan	23-50301-1	Scenario 1: Dynamic Coastal City	25%
Mehedi Hassan	22-49638-3	Scenario 2: Downtown Neon District	25%
Ashab Mahmud Tousif	24-56339-1	Scenario 3: Lakeside Park	25%
Shahriyar Lipu	24-56330-1	Scenario 4: River Front Market Life	25%

Table of Contents

1. Introduction 3

2. Related Works 5

3. Implementation 8

4. Screenshots 13

5. Code Availability 25

6. Future Work 25

7. Conclusion 26

1. Introduction

CityLife is a comprehensive, multi-scenario two-dimensional urban simulation developed using OpenGL and the GLUT (OpenGL Utility Toolkit) framework. The project showcases the creative and technical capabilities of computer graphics programming by rendering four distinct, richly detailed city environments within a single, cohesive application. Each scenario represents a unique urban setting—a dynamic coastal port city, a rain-soaked downtown neon district, a tranquil riverside park, and a festive winter night market—offering the viewer a varied and immersive visual journey through different times of day, seasons, and weather conditions.

The primary motivation behind CityLife is to demonstrate how traditional 2D rendering techniques, when combined with thoughtful animation, atmospheric shading, and layered compositing, can produce scenes that feel alive, dynamic, and visually engaging without relying on 3D engines or GPU shaders. In many computer graphics curricula, student projects tend to focus on static scenes or minimal interactivity; CityLife pushes beyond that boundary by incorporating continuous day-night cycles, real-time weather transitions (rain, snow, lightning storms), moving vehicles and pedestrians, animated wildlife (seagulls, butterflies, fish), water reflections with procedural ripples, particle-based exhaust and steam effects, and interactive keyboard controls that let the viewer switch between modes and trigger special events on demand.

The novelty of CityLife lies in several key areas. First, the four-scenario architecture uses C++ namespaces to encapsulate each scene's entire state—globals, structs, drawing functions, and timer callbacks—inside its own namespace (Scenario1_CoastalCity, Scenario2, Scenario3, Scenario4). This allows four team members to develop their scenes independently, with each person maintaining full creative control over their scenario's aesthetic and behavior, while the shared top-level framework handles navigation, pausing, and the HUD overlay seamlessly. Second, the project introduces a depth-scaling system (DepthScale) that applies perspective-like size variation to figures and objects based on their vertical position in the frame, creating an illusion of depth on a flat 2D canvas. Third, every scenario features a multi-layered water reflection system using matrix-based mirroring, alpha-blended wash overlays, and procedural ripple bands—a

technique typically associated with more advanced rendering pipelines but implemented here entirely through immediate-mode OpenGL calls.

Scenario 1, the Dynamic Coastal City, presents a bustling port with moving ships (cruise liners, cargo vessels, yachts, fishing boats), a helicopter patrol, a lighthouse beacon, vehicular traffic with braking logic and exhaust particle systems, pedestrians with walking animations and crosswalk behavior, seagull flocks, a glass elevator on a high-rise, and a full astronomical day-night cycle with sun, moon, and stars. Scenario 2, the Downtown Neon District, renders a moody, rain-drenched urban night with glowing neon signage, an elevated commuter train, wet-road reflections, a police car with flashing lights, a food truck with rising steam, a street performer, cyclists, and a detailed near-sidewalk with silhouetted foreground walkers. Scenario 3, the Riverside Park, depicts a peaceful parkland with cherry blossom trees (with falling petals), a Ferris wheel, a hot-air balloon, kayakers on a river, playground equipment (swings, seesaw, slide), a bandstand, a café terrace, a fairground, and wildlife including ducks, geese, squirrels, and butterflies. Scenario 4, the Winter Night Market, features a snowy plaza with a Christmas tree, ice-skating rink, market stalls with steam, a horse-drawn sled, a fire pit with ember particles, a clock tower with a swinging pendulum, carolers, fireworks, an aurora borealis, a frozen canal with boats, and full winter-to-autumn seasonal transitions.

The underlying platform is OpenGL 1.x with the FreeGLUT library, compiled using the GNU C++ toolchain (MinGW/GCC) with the C++11 standard. OpenGL's immediate-mode rendering (`glBegin/glEnd`) is used throughout for geometric primitives—triangles, quads, triangle fans, line strips, and points—while GLUT provides windowing, input handling (keyboard and mouse callbacks), and timer-driven animation at approximately 40 frames per second. The choice of this platform was deliberate: it keeps the entire project in a single, self-contained source file (approximately 19,700 lines of C++) with no external assets, textures, or model files, making it straightforward to compile and run on any Windows system with a standard OpenGL installation. Alpha blending (`GL_BLEND` with `GL_SRC_ALPHA / GL_ONE_MINUS_SRC_ALPHA`) is enabled globally, allowing translucent overlays, fog, glow effects, and smooth soft-edged shapes throughout all four scenarios.

2. Related Works

A number of existing projects have explored 2D city rendering, OpenGL-based scene simulation, and interactive computer graphics demonstrations. The following ten works represent notable efforts in this domain and have informed, directly or indirectly, the design decisions made in CityLife.

1. OpenGL City Scene by Ashish Kumar (GitHub)

URL: <https://github.com/AshishKumar4/OpenGL-City-Scene>

A 2D OpenGL project that renders a simplified city scene with buildings, roads, and vehicles using GLUT. It demonstrates basic transformations and immediate-mode rendering, similar to CityLife's foundational approach. However, it lacks dynamic weather, day-night cycling, and multi-scenario support, which CityLife introduces.

2. 2D City Simulation using OpenGL by Safin Singh (GitHub)

URL: <https://github.com/safinsingh/opengl-city>

This project models a flat city with scrolling backgrounds, simple traffic, and a basic sky gradient. It served as an early reference for structuring a multi-layered 2D scene, though CityLife extends the concept significantly with particle systems, reflection mechanics, and interactive controls.

3. Pixel City by Shamus Young

URL: <https://github.com/ShamusYoung/pixelcity>

Pixel City procedurally generates a 3D nighttime cityscape with glowing windows and light bloom effects. While it operates in 3D, its techniques for window lighting patterns and neon glow directly inspired CityLife's Scenario 2 (Downtown Neon District) and the approach to flickering window illumination across buildings.

4. OpenGL Rain and Snow Effects by ThinMatrix (YouTube Tutorial Series)

URL: <https://www.youtube.com/watch?v=EUGjguS21-4>

ThinMatrix's tutorial series demonstrates particle-based precipitation in OpenGL, covering rain streaks, snow accumulation, and splash effects. CityLife adapts similar particle approaches for its rain systems (Scenarios 1 and 2) and snow particles (Scenarios 1 and 4), though in a 2D immediate-mode context rather than the shader-based 3D pipeline shown in the tutorials.

5. Day-Night Cycle in 2D Games by Red Blob Games

URL: <https://www.redblobgames.com/articles/day-night/>

Red Blob Games presents an interactive article on implementing smooth day-night transitions using color interpolation and ambient overlays. CityLife's continuous day-night system (timeOfDay parameter cycling through noon, sunset, midnight, dawn) follows a similar linear interpolation strategy between predefined sky color presets.

6. GLUT-based 2D Animation Framework by Swiftless Tutorials

URL: <https://www.swiftless.com/tutorials/opengl/glut.html>

Swiftless provides a foundational guide to GLUT window creation, timer callbacks, and keyboard input handling. CityLife's timer-driven animation architecture (glutTimerFunc chains for each subsystem) follows the patterns described in these tutorials, extended to manage dozens of independent animation loops across four scenarios.

7. OpenGL 2D Water Reflections by Coding Train (Daniel Shiffman)

URL: <https://thecodingtrain.com/challenges/reflection>

Daniel Shiffman's coding challenges demonstrate mirroring and alpha-blended reflection techniques for 2D scenes. CityLife's BeginReflection/EndReflection system, which uses glScalef with a negative Y factor to flip geometry below a water surface, applies the same mathematical principle but adds procedural ripple bands and depth-fading wash overlays for greater realism.

8. Computer Graphics Through OpenGL by Sumanta Guha (Textbook)

URL: <https://www.sumantaguha.com/>

This widely-used textbook covers OpenGL fundamentals including transformations, projections, blending, and animation. It served as the primary academic reference for CityLife's rendering techniques, particularly the use of orthographic projection (glOrtho), alpha blending modes, and geometric primitive assembly.

9. Scenic Countryside OpenGL Project by University Projects Hub

URL: <https://github.com/topics/opengl-2d-scene>

A collection of university-level OpenGL 2D scene projects covering rural and urban environments. Several projects in this collection feature moving vehicles, scrolling clouds, and day-night toggles. CityLife differentiates itself through the scale of its

implementation (nearly 20,000 lines), the number of independent animated subsystems, and the seamless scenario-switching architecture.

10. Interactive Weather Visualization by W. Chen et al.

URL:

<https://ieeexplore.ieee.org/document/8456308>

This IEEE paper presents techniques for real-time weather visualization including rain, fog, and lightning rendering. CityLife implements comparable effects—procedural lightning bolts with branching geometry, fog layers with alpha gradients, and storm-intensity-driven sky color modulation—adapted for a 2D immediate-mode pipeline rather than the GPU shader approach described in the paper.

3. Implementation

3.1 Key Features

CityLife implements a broad set of interactive and visual features across its four scenarios. The most significant features are described below.

Dynamic Day-Night Cycle: Scenario 1 features a continuously cycling `timeOfDay` parameter (ranging from 0.0 to 1.0) that drives smooth color interpolation across five sky presets (Noon, Sunset, Night, Dawn, and Storm). The sun and moon follow sinusoidal orbital arcs, and the lighting factor adjusts building window illumination, street lamp activation, and vehicle headlight rendering accordingly. This system ensures that every visual element responds to the current time of day without any abrupt transitions.

Real-Time Weather System: Rain, snow, and lightning storms are simulated using particle arrays. Scenario 1 manages up to 1,000 simultaneous rain drops and 100 bubble particles for splash effects. Lightning is rendered as jagged `GL_LINE_STRIP` geometry with randomized node positions, accompanied by a full-screen flash overlay that modulates the sky gradient. Scenario 4 renders up to 1,500 snow particles across three depth bands (far, mid, near) with size scaling and wind drift.

Water Reflections and Ripples: A shared reflection system (`BeginReflection/EndReflection` with `WashReflection`) mirrors selected geometry below the water surface using a negative Y scale matrix transform. The reflected image is then

washed with the water's base color at increasing opacity with depth, and horizontal ripple bands are overlaid using sinusoidal offset calculations. This technique is used across Scenarios 1, 3, and 4.

Depth-Scaled Perspective: The `DepthScale` function maps a figure's Y position to a scale multiplier (0.62 at the horizon to 1.36 at the near ground), simulating perspective on a 2D plane. All pedestrians, vehicles, and ground-level objects are drawn using `BeginDepthSprite/EndDepthSprite` wrappers that apply this scaling around each object's anchor point.

Pedestrian and Traffic Systems: Each scenario maintains its own array of pedestrian structs with attributes for position, speed, walking phase, direction, and clothing colors. Vehicles have independent Update timer functions that handle position wrapping, braking logic at traffic lights, and exhaust particle emission. Scenario 1 includes seven pedestrians with crosswalk behavior and one dog-walking figure.

Interactive Controls: The application supports extensive keyboard interaction. Per-scenario controls (keys 1-4 for modes, R/T for weather in Scenario 2, W/A for wind and color in Scenario 3, S/L/F for season and lighting in Scenario 4) are handled through each scenario's own Keyboard callback, while global navigation (N/B for next/previous, F1-F4 for direct jump, H for help, Space for pause) is managed by the top-level keyboard and special functions. A HUD overlay system displays available controls contextually.

3.2 Key Technologies

The project is built entirely with OpenGL 1.x immediate-mode rendering and the FreeGLUT library, compiled with GCC (MinGW) using the C++11 standard (`-std=gnu++11`). Key technologies include: orthographic projection via `glOrtho` for the 120×80 world-unit coordinate system; alpha blending (`GL_SRC_ALPHA / GL_ONE_MINUS_SRC_ALPHA`) for translucent overlays, fog, and soft-edged ellipses; `glutTimerFunc` for frame-rate-independent animation with dozens of chained timer callbacks; `glPushMatrix/glPopMatrix` with `glScalef` and `glTranslatef` for depth-scaled sprites and water reflections; `GL_TRIANGLE_FAN` for circles and ellipses; `GL_LINE_STRIP` for lightning bolts and kite strings; bitmap font rendering via `glutBitmapCharacter` for the HUD and signage; and C++ namespaces for scenario

isolation, allowing four independent development streams to share a single translation unit without symbol collisions.

3.3 List of Key Functions

The following tables list the most important functions in each scenario of the project.

Scenario 1: Dynamic Coastal City (Sabit Hassan)

Function	Description
DrawSky()	Renders the dynamic sky gradient with day-night interpolation, sun, moon, and lightning
UpdateSun()	Timer callback that advances timeOfDay, stormFactor, and beacon pulse
DrawStars()	Renders fixed star positions during nighttime
DrawCloud1()–DrawCloud4()	Render four cloud formations with weather-modulated colors
DrawMountain1()–DrawMountain3()	Draw layered mountain silhouettes on the horizon
DrawPerson()	Renders an articulated pedestrian with arms, legs, walking animation, and optional dog
DrawHelicopter()	Draws the helicopter with animated rotor, searchlight, and tail rotor
DrawGlassElevator()	Animates a glass elevator on a high-rise building
DrawTrafficLight()	Renders traffic signals with red/yellow/green state logic
DrawWaveLayer()	Draws sinusoidal ocean waves with parallax scrolling
DrawRipples()	Renders expanding ring ripples on the ocean surface
DrawFog()	Overlays translucent fog bands during storm conditions
EmitExhaustSmoke()	Spawns exhaust particles behind moving vehicles
UpdateTrain() / UpdateCarRed() / UpdateBus()	Timer callbacks driving vehicle positions and braking
UpdateCruiseShip() / UpdateYacht()	Timer callbacks for ocean vessel movement
UpdatePedestrians()	Advances pedestrian positions, crosswalk logic, and walking phase
UpdateSeagulls()	Animates seagull flock positions and wing

	flapping
Init()	Initializes all Scenario 1 state and registers timer callbacks
Draw()	Master draw function compositing all Scenario 1 elements back-to-front

Scenario 2: Downtown Neon District (Mehedi Hassan)

Function	Description
DrawSky2()	Renders the dusk/night sky with exposure-based tinting
DrawSkyline2()	Draws a distant city skyline silhouette
DrawTrain2() / UpdateTrain2()	Elevated commuter train with coaches and animation
DrawHelicopter2() / UpdateHelicopter2()	Police helicopter with searchlight sweep
DrawBuildingBody2() / DrawWindows2()	Renders skyscrapers with flickering lit windows
DrawNeonSign2() / DrawAllNeon2()	Pulsating neon signage on building facades
DrawPuddleGlow2()	Neon reflections in wet road puddles
DrawLightningBolt2() / SpawnLightningBolt2()	Procedural lightning with branching forks
DrawVehicles2() / UpdateVehicles2()	Multi-type vehicle system (taxi, bus, van, limo, motorcycle)
DrawPoliceCar2()	Police car with alternating red-blue flashing lights
DrawBicycle2()	Animated cyclist with pedaling mechanics
DrawRain2() / UpdateRain2()	Rain particle system with wind-angled streaks
DrawWetRoad2()	Reflective wet road surface with streaked highlights
DrawNearSidewalk2()	Detailed foreground sidewalk with drain, grate, litter
DrawNearWalker2()	Large-scale silhouette figures on the near sidewalk
DrawStreetPerformer2()	Animated busker with arm waving
DrawFoodTruck2() / DrawSteam2()	Food truck with rising steam puff particles
DrawApartmentWindow2()	Animated interior scene visible through an apartment window
Draw()	Master compositing function for all Scenario 2 elements

Scenario 3: Lakeside Park View (Ashab Mahmud Tousif)

Function	Description
DrawSky3() / DrawSun3()	Sky gradient and sun with halo for four time-of-day phases
DrawClouds3() / UpdateClouds3()	Scrolling cumulus cloud formations
DrawBalloon3() / UpdateBalloon3()	Hot-air balloon with basket, ropes, and altitude bobbing
DrawJet3() / UpdateJet3()	Flyover jet with contrail
DrawRiver3() / UpdateRiver3()	Multi-layered river with current, reflections, and ripples
DrawKayak3() / DrawKayakBody3()	Detailed kayak with animated paddler and stroke mechanics
DrawDucks3() / DrawGooseFamily3()	Wildlife: ducks, goose family, duck family on the river
DrawFountain3() / UpdateFountain3()	Park fountain with animated water jets
DrawSwing3() / DrawSeesaw3() / DrawSlide3()	Playground equipment with child figures
DrawCafe3()	Multi-table café terrace with seated patrons and umbrellas
DrawBandstand3()	Musical bandstand with performer figures and instruments
DrawFerrisWheel3()	Ferris wheel with rotating cabins
DrawFairground3()	Fairground area with ticket booth, bunting, and queue
DrawBranch3() / DrawTree3()	Recursive tree rendering with seasonal leaf colors
DrawPetals3()	Falling cherry blossom / autumn leaf particle system
DrawPerson3()	Articulated pedestrian with 14 clothing/accessory variations
DrawFireflies3() / DrawButterflies3()	Ambient wildlife animations
Draw()	Master compositing function for the park scene

Scenario 4: River Front Market Life (Shahriyar Lipu)

Function	Description
DrawSky4() / DrawMoon4() / DrawStars4()	Night sky with moon, twinkling stars, and bright star crosses

DrawBirds4() / DrawSleigh4()	Flying birds and animated Santa's sleigh
DrawShop4() / DrawShops4()	Market shop facades with lit windows and signage
DrawFerrisWheel4() / UpdateFerrisWheel4()	Large Ferris wheel with colored cabins
DrawRink4() / DrawSkater4()	Ice skating rink with animated skater figures
DrawStall4() / DrawStalls4()	Market stalls with goods, awnings, and customer queues
DrawChristmasTree4()	Decorated Christmas tree with ornaments and star topper
DrawFirePit4() / UpdateFire4()	Fire pit with ember particle system
DrawSnowman4() / DrawSnowmanFamily4()	Snowman figures with accessories
DrawSanta4()	Santa Claus figure with sack
DrawClockTower4() / UpdatePendulum4()	Clock tower with swinging pendulum animation
DrawFireworks4() / UpdateFireworks4()	Firework burst particle system with ground flash
DrawAurora4() / UpdateAurora4()	Aurora borealis with sinusoidal color bands
DrawPerspectiveRoad4()	Converging-perspective road with vehicles and lampposts
DrawCar4() / DrawBike4() / DrawRickshaw4()	Street vehicles with animation
DrawRiver4() / DrawBoats4() / DrawFrozenCanal4()	River and canal with boats and ice
DrawSnowBand4() / UpdateSnow4() / InitSnow4()	Three-layer snow particle system
DrawTeaStall4()	Detailed tea stall with seated customers and steam
UpdateSeason4()	Winter-to-autumn seasonal transition system
Draw()	Master compositing function for the winter market scene

4. Screenshots of the Project

Scenario 1: Dynamic Coastal City

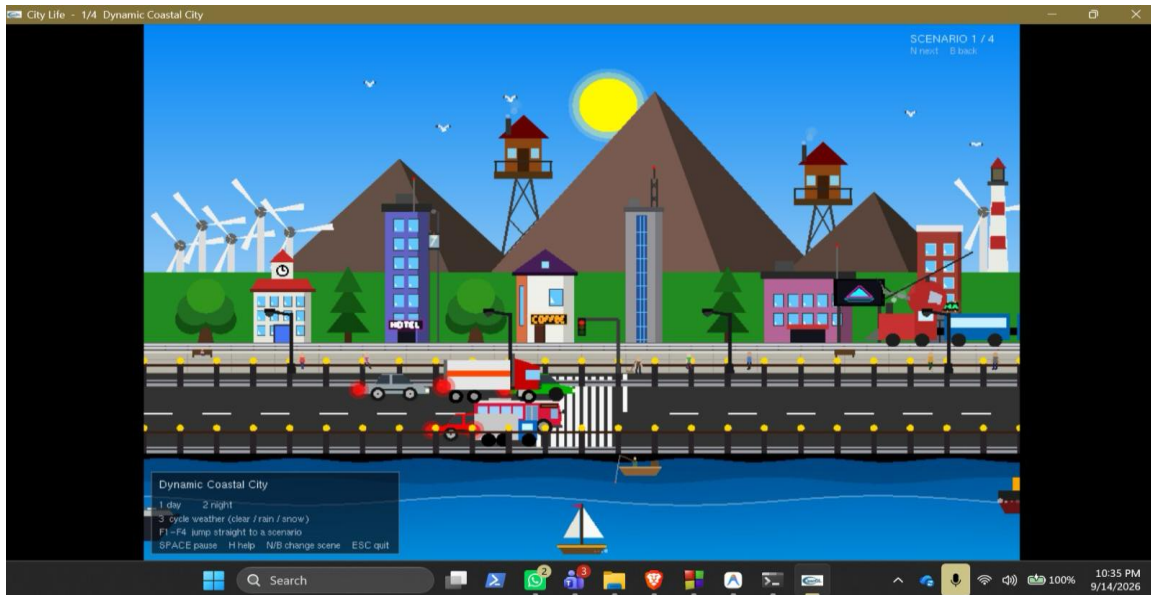


Figure 1: Daytime view with sun, clouds, moving vehicles, and ocean

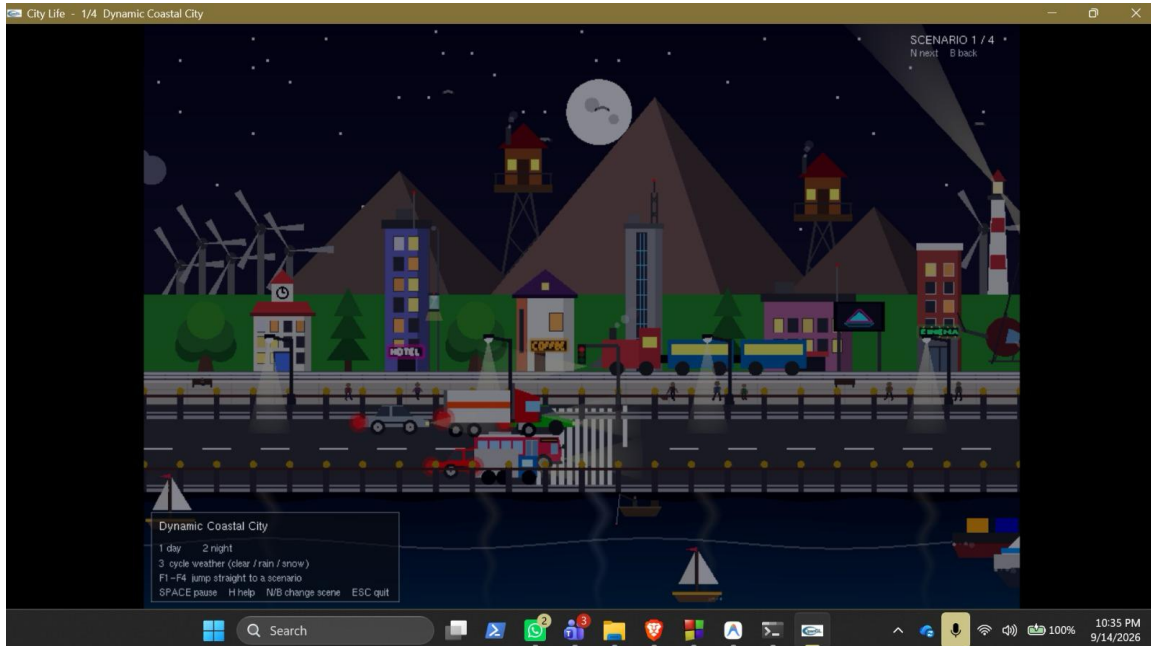


Figure 2: Night scene with moon, stars, lit buildings, and lighthouse beacon

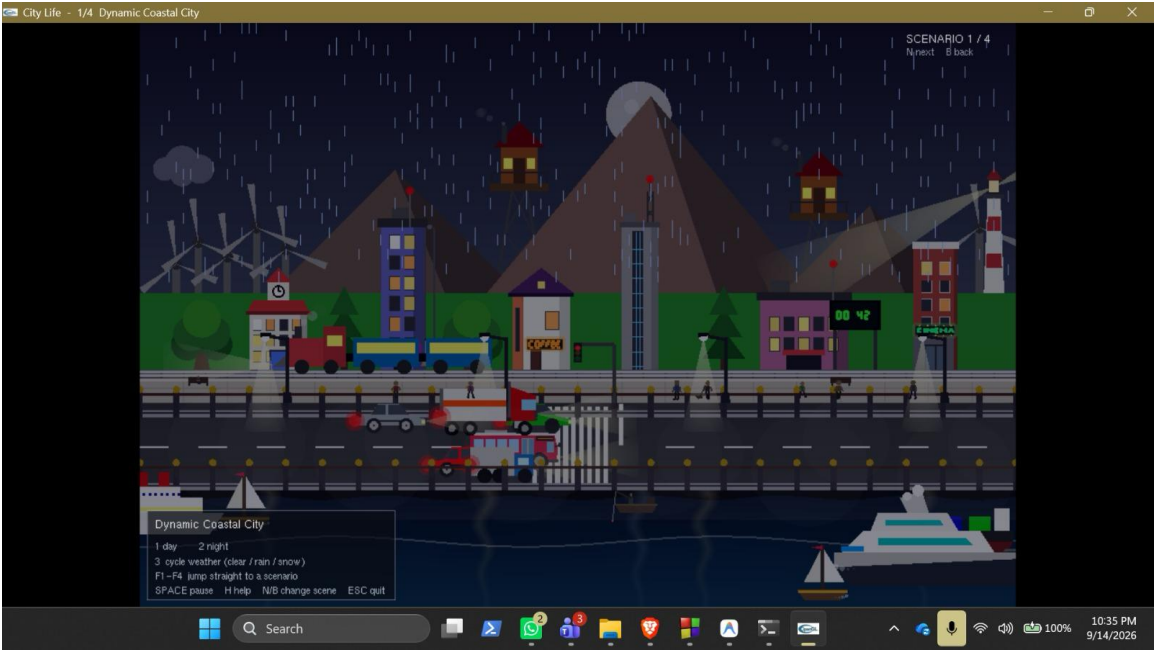


Figure 3: Storm weather with rain, lightning, and dark sky

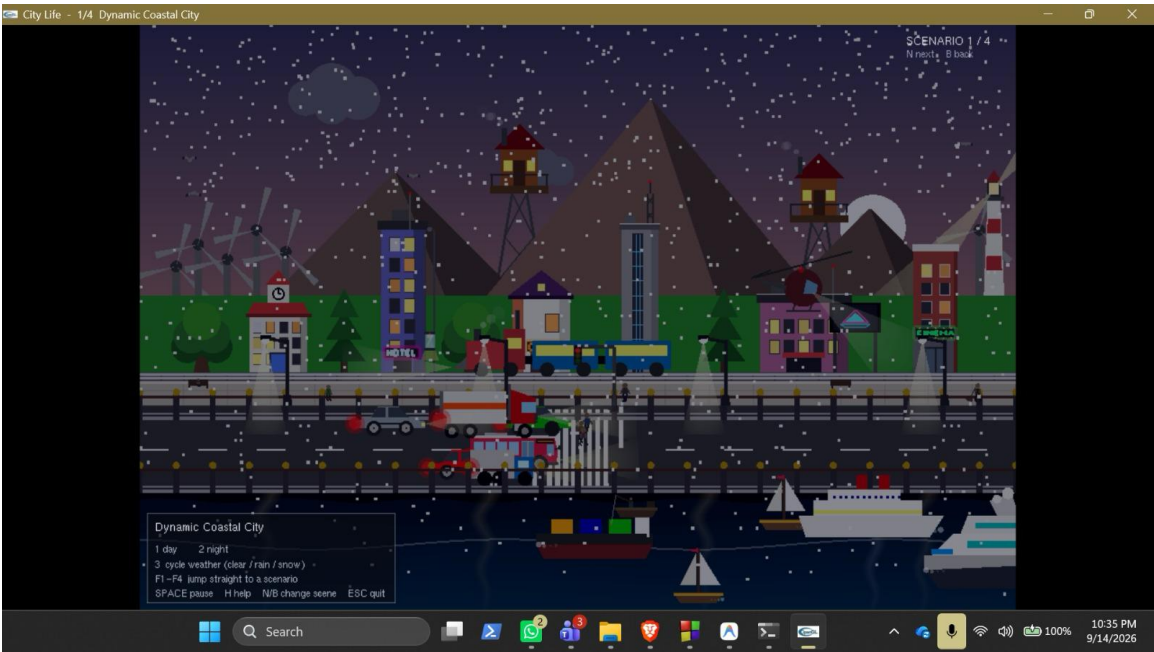


Figure 4: Helicopter patrol over the coastal port



Figure 5: Ocean with cruise ship, yacht, and water reflections

Scenario 2: Downtown Neon District



Figure 6: Neon-lit downtown street at night with wet road reflections



Figure 7: Elevated train passing through the skyline



Figure 8: Rain storm with lightning bolt and puddle glow



Figure 9: Near sidewalk with silhouette walkers and food truck



Figure 10: Police car with flashing lights and street traffic

Scenario 3: Riverside Park

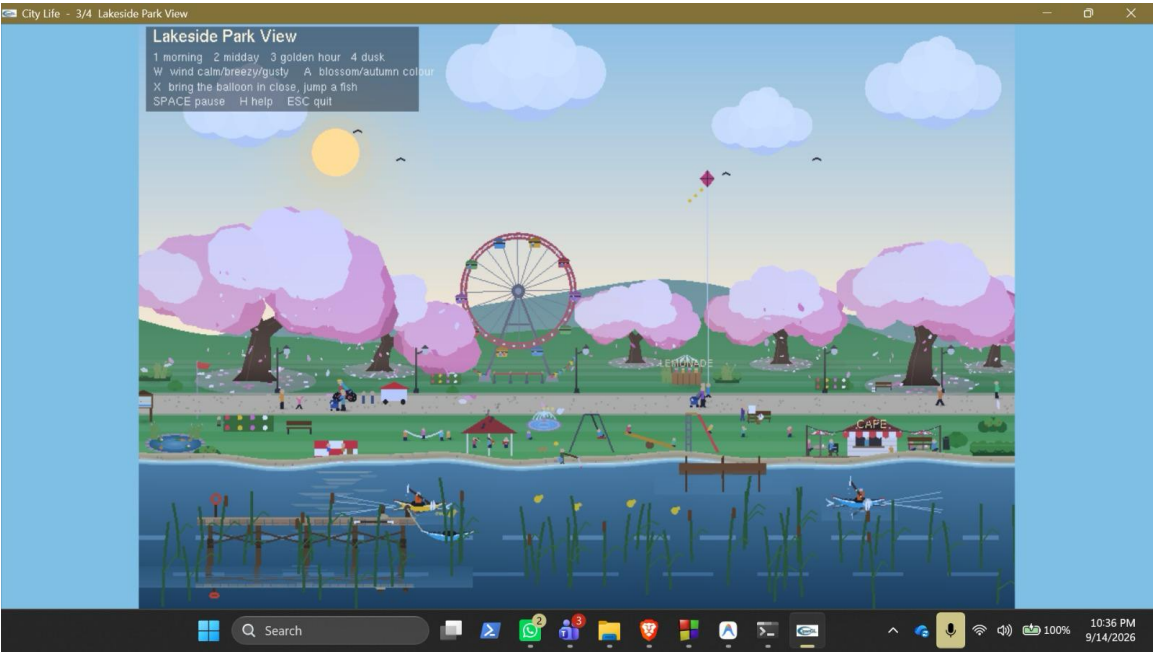


Figure 11: Morning riverside park with cherry blossoms and kayaker

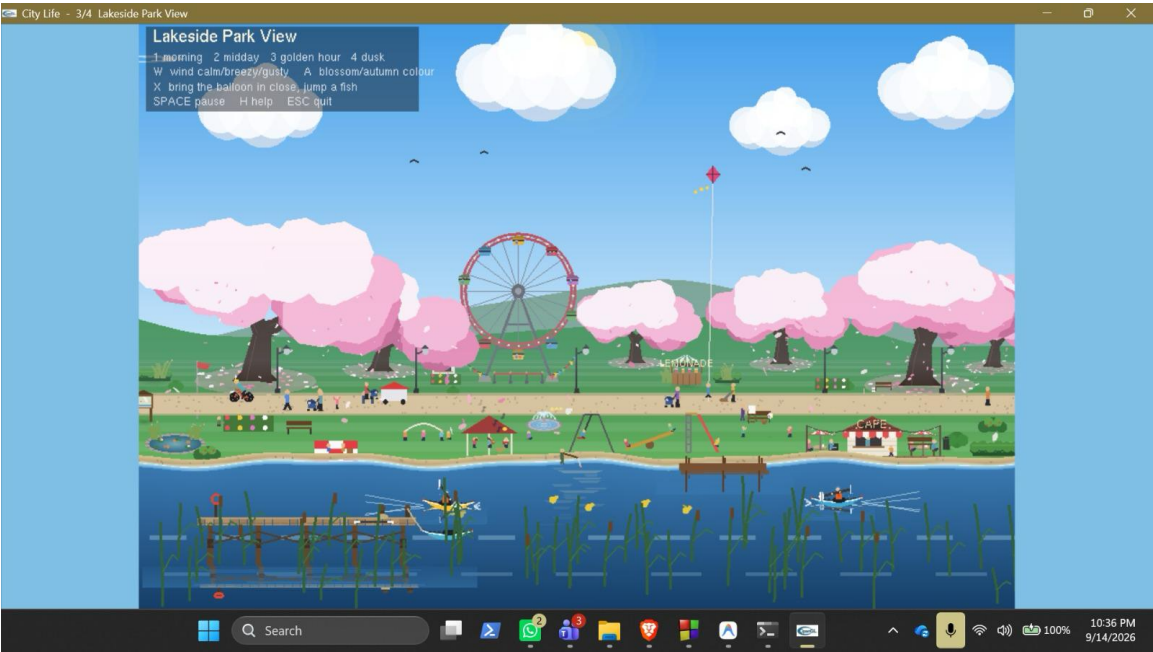


Figure 12: Ferris wheel and fairground area

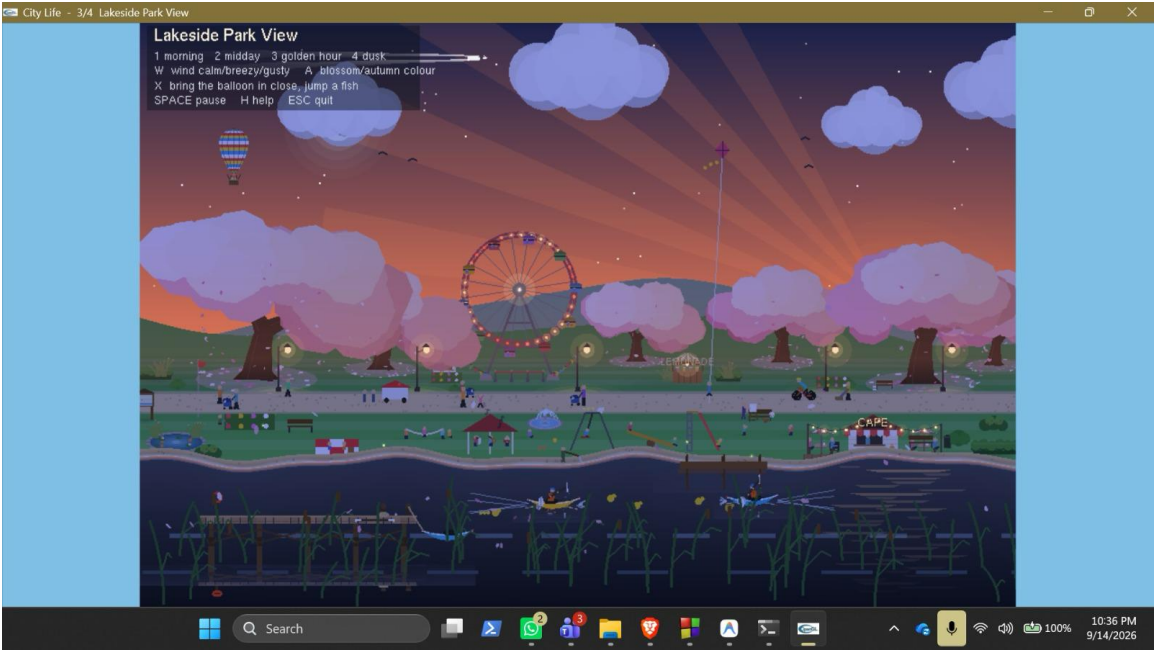


Figure 13: Hot-air balloon over the river with ducks and geese

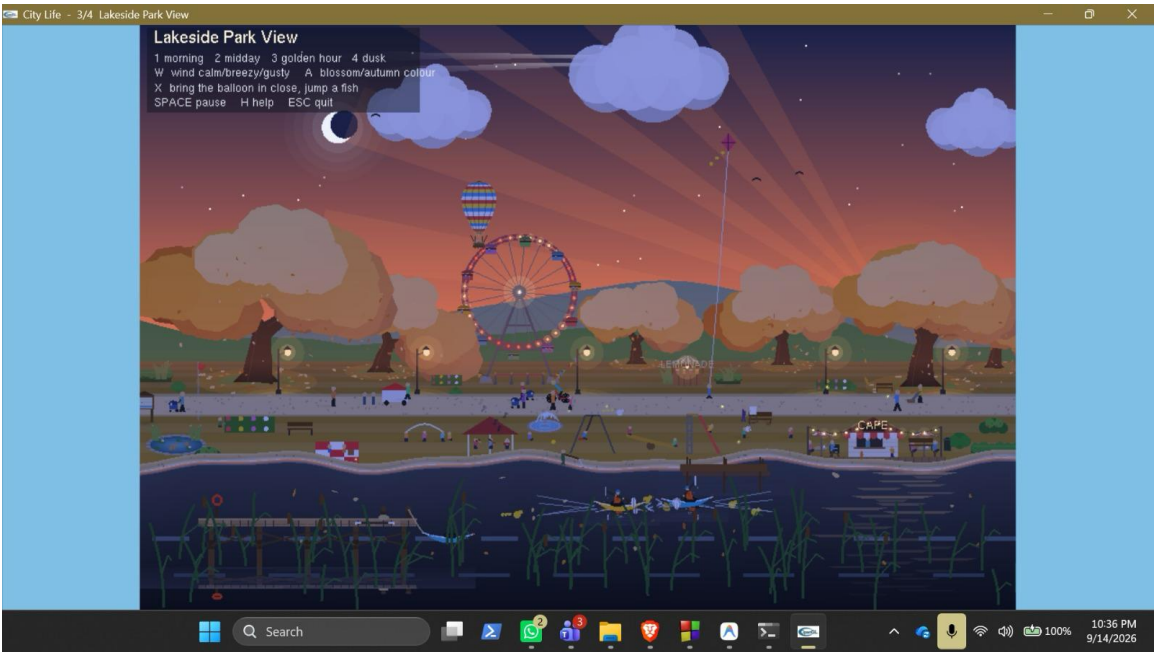


Figure 14: Playground area with swings, seesaw, and children

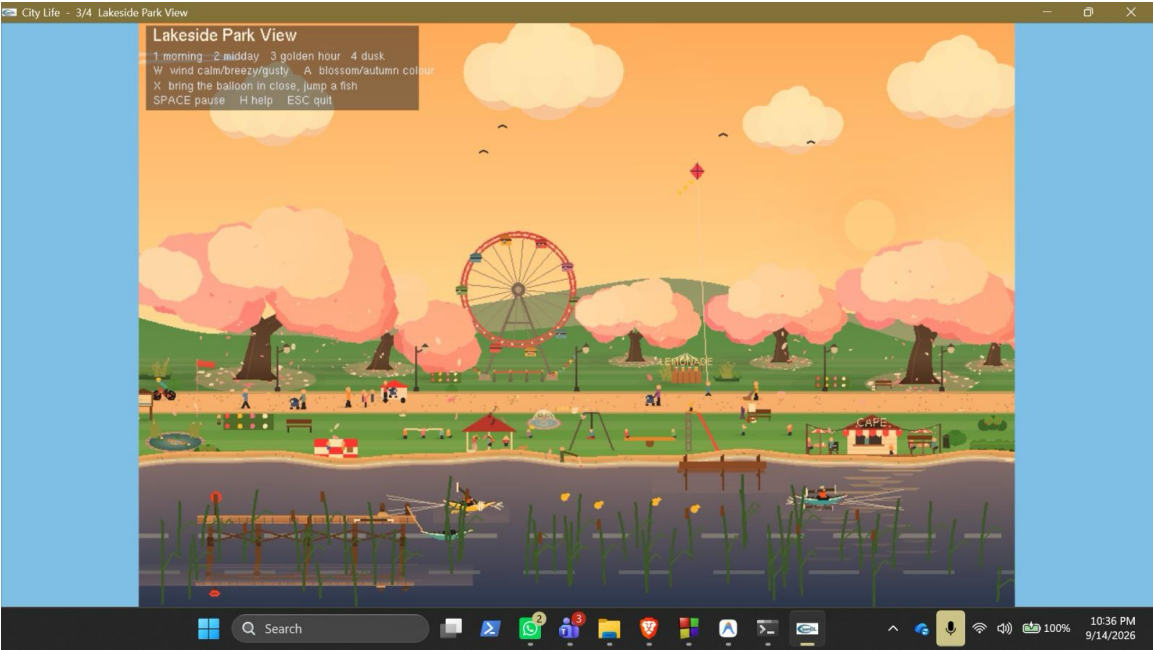


Figure 15: Golden hour scene with bandstand and café terrace

Scenario 4: Winter Night Market

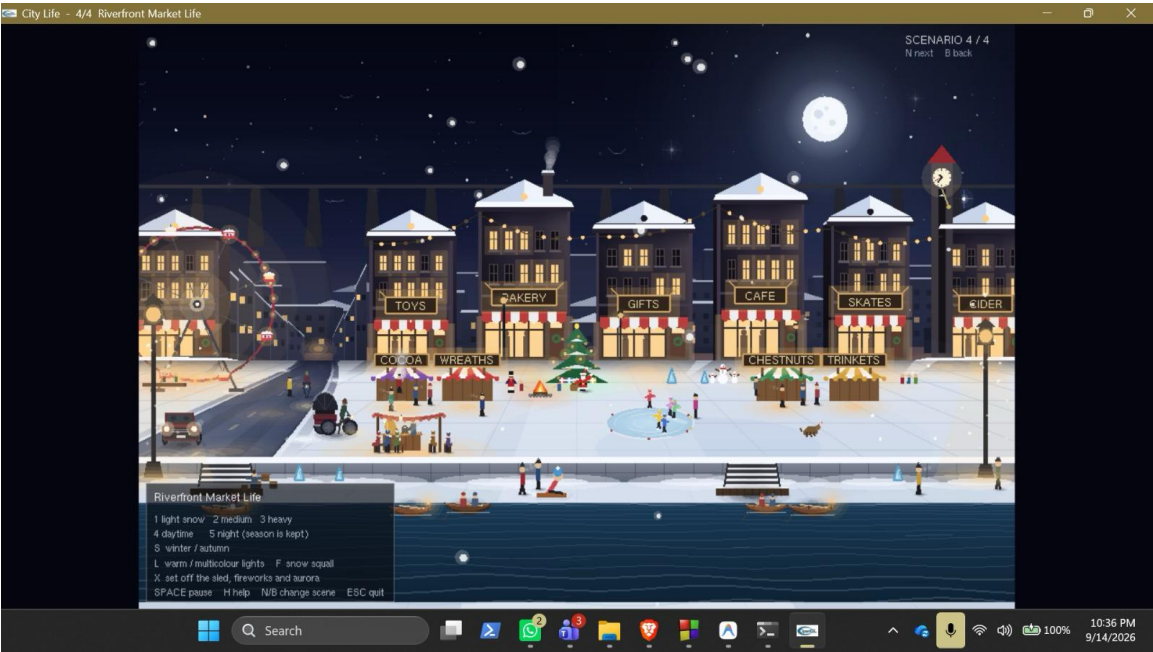


Figure 16: Winter night market plaza with Christmas tree and stalls



Figure 17: Autumn Ferris wheel, aurora and ice-skating rink at night

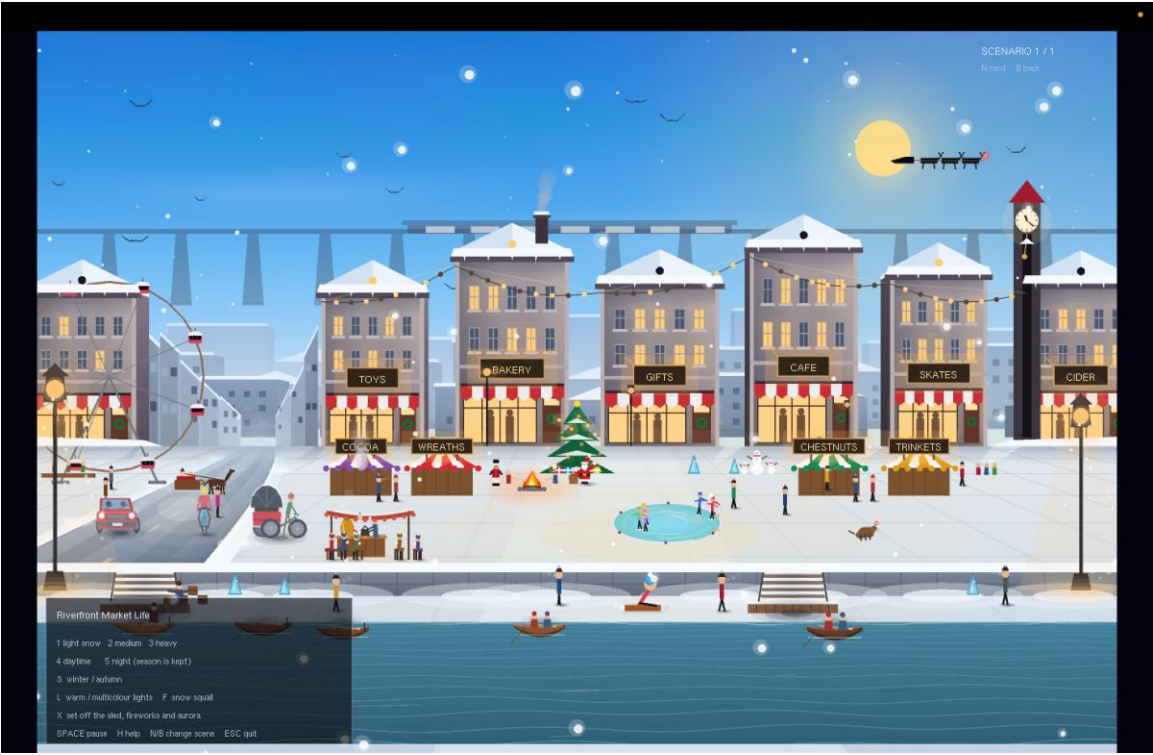


Figure 18: Autumn Daylight over the market

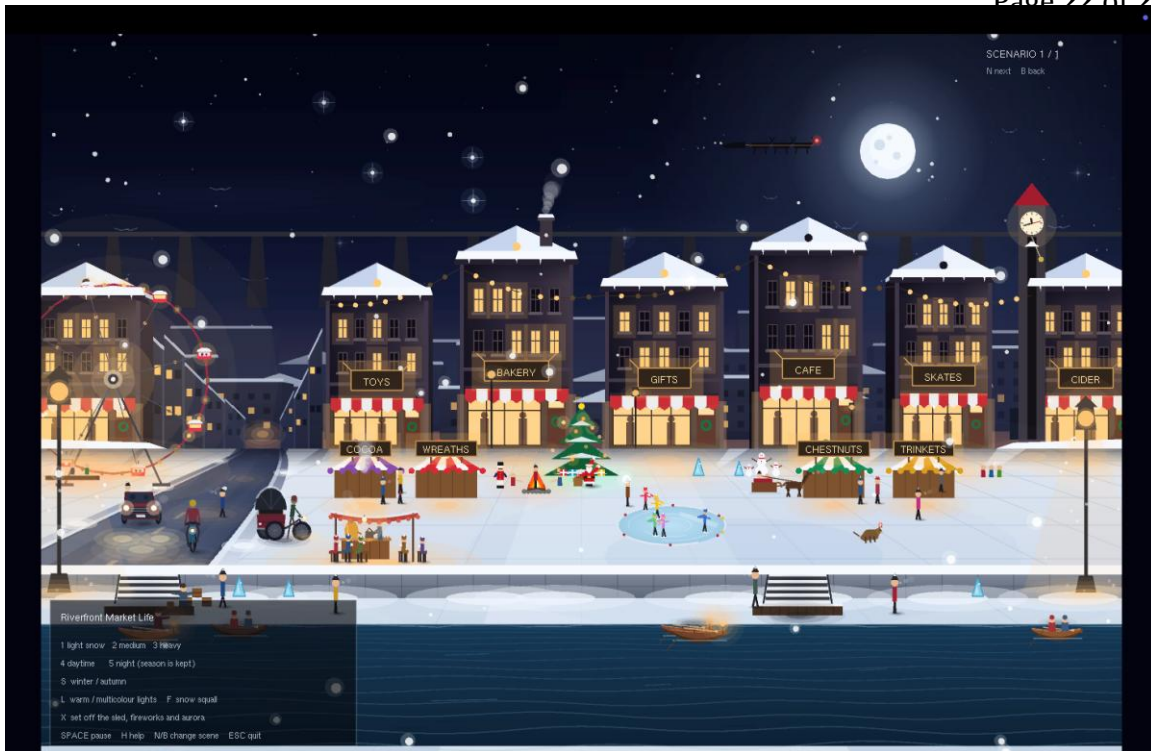


Figure 19: Winter river with boats and snow falling



Figure 20: Autumn season transition with fog and warm lighting

Graph Plotting:

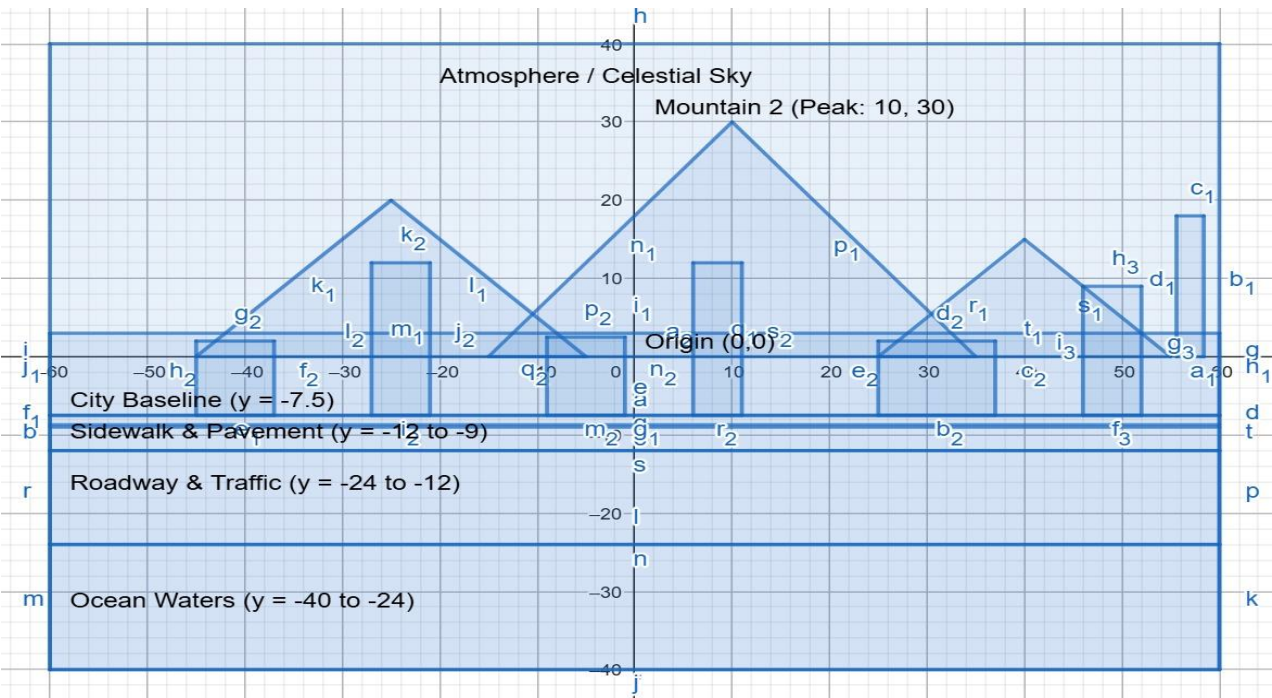


Figure 21: Graph plotting Scenario 1

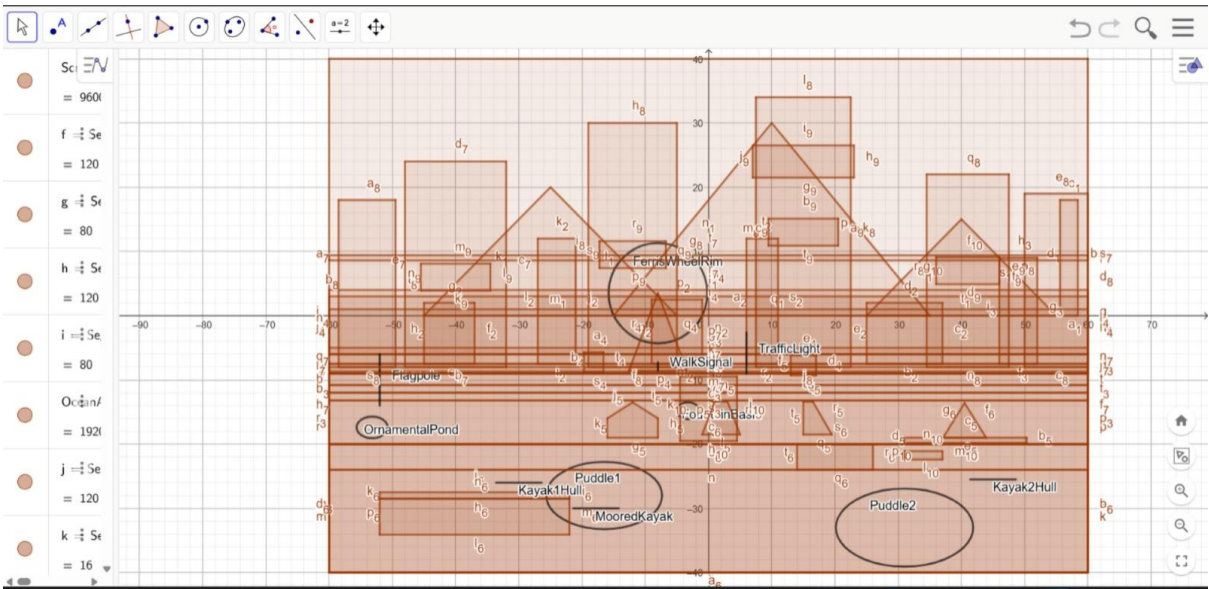


Figure 22: Graph plotting Scenario 2

Figure 23: Graph plotting Scenario 3

5. Code Availability

The complete source code of the CityLife project is publicly available on GitHub. The repository contains the single-file C++ source (CityLifeFinalFile.cpp, approximately 19,700 lines) along with build instructions for Code::Blocks and command-line compilation with MinGW/GCC.

GitHub Repository:

https://github.com/shahriyar-lipu/Computer-Graphics_Open-GL_City-Life

To compile the project, ensure that the C++11 standard is enabled (-std=gnu++11) and link against FreeGLUT, OpenGL, and GLU libraries (-lfreeglut -lopengl32 -lglu32). The project has been tested on Windows with MinGW GCC and Code::Blocks IDE.

6. Future Work

The following enhancements are planned or envisioned for future iterations of the CityLife project:

- **Texture Mapping and Sprite Integration:** Replace procedural polygon-based rendering of buildings, vehicles, and characters with texture-mapped sprites to improve visual fidelity and reduce the vertex count per frame.
- **Shader-Based Rendering Pipeline:** Migrate from OpenGL 1.x immediate mode to a modern OpenGL 3.3+ pipeline with vertex/fragment shaders, enabling GPU-accelerated post-processing effects such as bloom, motion blur, and screen-space reflections.
- **Audio Integration:** Add ambient soundscapes (ocean waves, rain, city traffic, market chatter) and event-driven sound effects (lightning thunder, firework bursts, train whistles) using an audio library such as OpenAL or FMOD.
- **Procedural City Generation:** Implement algorithmic building placement and road layout generation so that each run produces a unique city layout, increasing replay value and demonstrating procedural content generation techniques.

- **User-Controlled Camera and Zoom:** Add mouse-driven panning and scroll-wheel zoom to let the viewer explore each scenario freely, rather than viewing a fixed frame.
- **Multiplayer / Networked Mode:** Allow multiple users to view the same city simultaneously, with each user controlling a different camera or avatar within the scene.
- **Mobile and Web Deployment:** Port the project to WebGL (via Emscripten) or OpenGL ES for mobile platforms, making the simulation accessible through a web browser or smartphone.
- **AI-Driven Pedestrian Behavior:** Replace the current scripted pedestrian paths with a simple AI system that allows figures to make navigation decisions, avoid obstacles, and interact with each other and the environment dynamically.

7. Conclusion

7.1 Limitations

- **Single-Threaded Rendering:** The entire application runs on a single CPU thread with immediate-mode OpenGL calls. As the scene complexity grows (particularly Scenario 4 with its snow, fireworks, and aurora systems running simultaneously), frame rates may drop on older hardware. A modern retained-mode or GPU-accelerated pipeline would address this.
- **No Texture Support:** All visual elements are rendered purely from geometric primitives (polygons, lines, points). While this keeps the project dependency-free, it limits the visual richness compared to texture-mapped scenes and means that detailed surfaces (brick walls, wood grain, fabric patterns) cannot be represented.
- **Fixed Resolution Assumptions:** Although the reshape callback adjusts the viewport, some bitmap text positioning and HUD layout calculations assume the default 1200×800 window size. Extreme window aspect ratios may cause minor layout misalignment.

- No Audio: The project is entirely silent. Weather events like thunder, ambient city sounds, and musical performances in the bandstand would significantly enhance immersion but are not implemented.
- Platform Dependency: The project targets Windows (windows.h, -lopengl32, -lglu32) and requires MinGW with FreeGLUT. Cross-platform compilation for macOS or Linux would require conditional includes and different linker flags.

7.2 Summary

CityLife successfully demonstrates that a single self-contained C++ source file, using only the OpenGL and GLUT libraries, can produce a visually rich, interactive, and multi-faceted urban simulation. The project's four scenarios—Dynamic Coastal City, Downtown Neon District, Riverside Park, and Winter Night Market—collectively showcase a wide spectrum of computer graphics techniques: continuous day-night cycling, real-time weather simulation, particle-based effects, procedural water reflections, depth-scaled perspective, articulated character animation, and interactive keyboard controls.

Each team member contributed a complete, standalone scenario with its own unique aesthetic, animation systems, and interactive features, while the shared namespace-based architecture ensured seamless integration. The final deliverable, at nearly 20,000 lines of C++, represents a significant engineering effort and serves as both a technical demonstration and a creative showcase of what can be achieved with foundational computer graphics programming techniques. The project lays a strong foundation for future enhancements including shader-based rendering, audio integration, and procedural content generation.